Вернуться к 'Функции'

Тест начат	Понедельник, 21 Сентябрь 2026, 17:56
Состояние	Завершенные
Завершен	Понедельник, 21 Сентябрь 2026, 18:29
Прошло времени	33 мин. 6 сек.
Баллы	11,00/11,00
Оценка	**5,00** из 5,00 (**100**%)

Вопрос **1**

Верно

Баллов: 1,00 из 1,00

Выберите правильную реализацию функции `remove :: Int -> [a] -> [a]`, которая удаляет элемент из списка с позиции n.

Например , `remove 0 [1, 2, 3, 4] = [2, 3, 4]`

Выберите один ответ:

- `remove n xs = take n xs ++ drop n xs`
- `remove n xs = take n xs ++ drop (n + 1) xs` ✔
- `remove n xs = drop n xs ++ take n xs`
- `remove n xs = take (n + 1) xs ++ drop n xs`

Вопрос **2**

Верно

Баллов: 1,00 из 1,00

Какое из следующих определений являются правильным для логического оператора конъюнкции && (т.е. И)? Выберите все реализации, которые дают правильный ответ, основанный на таблице истинности.

Примечание: так как && оператор уже определен в Prelude, вы можете скрыть реализацию по умолчанию, чтобы проверить следующий код. Для этого добавьте следующую строку в ваш код:

```
import Prelude hiding ((&&))
```

Выберите один или несколько ответов:

- [ ] `a && b = if not (a) then not (b) else True`
- [x] `a && b = if a then b else False` ✔
- [ ] `a && b = if a then b`
- [x] `True && True = True`
  `&& = False` ✔
- [ ] `a && b = if a then if b then False else True else False`
- [x] `a && b = if a then if b then True else False else False` ✔
- [x] `a && b = if b then a else False` ✔

Вопрос **3**

Верно

Баллов: 1,00 из 1,00

Укажите тип : `f xs = take 3 (reverse xs)`

Выберите один ответ:

- (•) [a] -> [a] ✔
- ( ) Int-> [a]
- ( ) Int -> [a] -> [a]
- ( ) Int
- ( ) [a]

Вопрос **4**

Верно

Баллов: 1,00 из 1,00

Укажите, как каррированная функция `mult x y z = x * y * z` может быть понята с точки зрения лямбда-выражений.

Выберите один или несколько ответов:

- [x] `mult = \ x -> (\ y -> (\ z -> x * y * z))` ✔
- [ ] `mult = \ x -> (x * \ y -> (y * \ z -> z))`
- [ ] `mult = ((((\x -> \y) -> \z) -> x * y) * z)`
- [ ] `mult x y z = \ x -> (\ y -> (\ z -> x * y * z))`

Вопрос **5**

Верно

Баллов: 1,00 из 1,00

Какое из следующих определений являются правильным для логического оператора дизъюнкции || (T.e. OR)? Выберите все правильные реализации!

Примечание: так как || оператор уже определен в Prelude, вы можете скрыть реализацию по умолчанию, чтобы проверить следующий код. Для этого добавьте следующую строку в ваш код:

```
import Prelude hiding ((||))
```

Выберите один или несколько ответов:

- [x]
```
b || False = b
_ || True = True
```
✔

- [ ]
```
b || c
  | b == c = True
  | otherwise = False
```

- [x]
```
False || b = b
True || _ = True
```
✔

- [x]
```
False || False = False
False || True = True
True || False = True
True || True = True
```
✔

- [x]
```
b || c
  | b == c = c
  | otherwise = True
```
✔

- [ ]
```
b || True = b
_ || True = True
```

- [x]
```
False || False = False
_ || _ = True
```
✔

- [x]
```
b || c
  | b == c = b
  | otherwise = True
```
✔

Вопрос **6**

Верно

Баллов: 1,00 из 1,00

Haskell.

Укажите тип данной функции: `pair x y = (x, y)`

Выберите один ответ:

- a -> b -> c
- (a, b) -> (a, b)
- a -> b -> a -> b
- a -> b -> (a, b) ✔

Вопрос **7**

Верно

Баллов: 1,00 из 1,00

Какие из следующих реализаций верные для функции safetail :: [a] -> [a], которая похожа на функцию tail, за исключением того, что safetail возвращает пустой список, если исходный список пуст, в то время как стандартная функция tail выдает ошибку в этом случае. Выберите все правильные реализации!

Выберите один или несколько ответов:

- [x] ✔
```
safetail xs
  | null xs = []
  | otherwise = tail xs
```
- [ ]
```
safetail xs = tail xs
safetail [] = []
```
- [x] ✔
```
safetail
  = \ xs ->
      case xs of
          [] -> []
          (_ : xs) -> xs
```
- [ ]
```
safetail [x] = [x]
safetail (_ : xs) = xs
```
- [x] ✔
```
safetail [] = []
safetail (_ : xs) = xs
```
- [x] ✔
```
safetail [] = []
safetail xs = tail xs
```
- [x] ✔
```
safetail xs = if null xs then [] else tail xs
```
- [ ]
```
safetail (_ : xs)
  | null xs = []
  | otherwise = tail xs
```

Вопрос **8**

Верно

Баллов: 1,00 из 1,00

Укажите тип данной функции: `second xs = head (tail xs)`

Выберите один ответ:

- [ ] [a]
- [ ] [a] -> [a]
- [x] [a] -> a ✔
- [ ] [(Bool, Int)]

Вопрос **9**

Верно

Баллов: 1,00 из 1,00

Укажите правильные реализации функции `halve :: [a] -> ([a], [a])` , который разбивает список четной длины на две половины? Выберите все правильные реализации!

Выберите один или несколько ответов:

- [x]
```
halve xs = (take (n `div` 2) xs, drop (n `div` 2) xs)
  where n = length xs
```
- [x]
```
halve xs = (take n xs, drop n xs)
  where n = length xs `div` 2
```
- [ ]
```
halve xs = (take n xs, drop n xs)
  where n = length xs / 2
```
- [x]
```
halve xs = splitAt (div (length xs) 2) xs
```
- [ ]
```
halve xs = splitAt (length xs / 2) xs
```
- [ ]
```
halve xs = (take n xs, drop (n + 1) xs)
  where n = length xs `div` 2
```
- [x]
```
halve xs = splitAt (length xs `div` 2) xs
```
- [ ]
```
halve xs = splitAt (length xs `div` 2)
```

Вопрос **10**

Верно

Баллов: 1,00 из 1,00

Укажите тип данной функции: `swap ( x, y) = (y, x)`

Выберите один ответ:

- (a, b) -> (b, a) ✔
- Tuple a => a -> a
- (a, b) -> (a, b)
- (a, a) -> (a, a)

Вопрос **11**

Верно

Баллов: 1,00 из 1,00

Укажите правильный вывод вызова функции `funct 3 [1, 2, 3, 4, 5, 6, 7]`?
функция опрделена как ,

```
funct :: Int -> [a] -> [a]
funct x xs = take (x + 1) xs ++ drop x xs
```

Выберите один ответ:

- [1, 2, 3, 4, 4, 5, 6, 7] ✔
- [1, 2, 3, 5, 6, 7]
- [1, 2, 3, 4, 4, 6, 7]
- [4, 5, 6, 7, 1, 2, 3]